

Hawrilenko Poker Strategy as a Frontier Policy for a Quantum Web Crawler

Executive summary

Matt Hawrilenko's most durable strategic ideas are not flashy hand-reading tricks. They are a disciplined sequence: know your own distribution, keep it balanced, make opponents indifferent where possible, and then exploit only at the margins, with the size of the exploit bounded by the strength of the read. In his MIT decision-making lecture, he repeatedly argues that players should stop trying to “do magic” with precise opponent-hand guesses and instead reason over distributions, frequencies, and how each action reshapes the rest of the strategy. He also emphasizes that downstream weaknesses can often be exploited earlier in the hand by altering pot size before the obvious exploit point arrives. Those ideas translate unusually well to web-crawler frontier management, because a crawler also operates under partial observability, strategic feedback, and scarce budget. ¹

The core translation is this: treat every frontier URL not as a single deterministic object, but as a **range** over hidden future states across the next n hops. Those states include “valuable,” “fresh,” “duplicate,” “trap,” “blocked,” “slow,” “authoritative,” and “stale.” Hawrilenko’s “read your own hand” becomes “know your own frontier distribution”: estimate where a candidate sits in the crawler’s ranked posterior over future value and risk, and make decisions based on percentile, balance, and opportunity cost rather than isolated anecdotes. His pot-odds logic becomes a continuation threshold: how much expected value or uncertainty reduction must a candidate provide, relative to crawl cost and delay, to justify spending one more request. His exploitative play becomes bounded site-specific adaptation: move only the marginal URLs up or down unless the evidence that a site is systematically underexposed, overfarmed, trap-prone, or unusually rich becomes strong. ²

A practical architecture follows naturally. Use a classical frontier manager with a host-aware politeness scheduler inspired by Mercator, because real crawlers must maintain bounded memory, host-level rate control, and robots compliance. On top of that base, maintain a multi-task predictive model that estimates n -hop cumulative value, freshness hazard, structural novelty, authority, duplication risk, latency, and adversarial-trap hazard. Then apply a two-stage policy: a calibrated **GTO-like base policy** that is hard to exploit and respects cost/risk constraints, followed by a small **exploit adjustment** that reacts to site-specific evidence with margin-limited confidence scaling. A quantum component is most credible today as a **small-set reranker** or graph-subgraph scorer over the top- K candidates, not as the primary internet-scale frontier engine. The nearest primary literatures are quantum PageRank, quantum SearchRank, quantum kernels, quantum graph neural networks, and hybrid temporal graph models for link prediction—not production quantum crawling systems. ³

The resulting algorithm should optimize the **area under the harvest curve** rather than only final coverage. Foundational crawler work showed that URL ordering matters because limited crawls rarely reach the whole web, and that focused/frontier methods improve the rate at which important pages are found. Modern neural frontier work further shows that semantically informed best-first policies can improve early-stage

harvest rate and ranking metrics on ClueWeb22-based experiments. The right evaluation, therefore, compares how fast the system discovers valuable information, how fresh that information remains, how much cost it incurs, and how robust it is to duplicate farms, faceted traps, slow hosts, and adversarial feedback. ⁴

Poker principles translated into crawler control

Hawrilenko's most useful abstraction for the crawler is that **single actions should be judged by how they reshape the whole distribution**, not by whether an isolated guess feels clever. In poker, he stresses that if you change what you do with one part of your range, you implicitly change the rest. In crawling, that means unit decisions about one URL affect the visible frontier composition, the remaining host budgets, the freshness backlog, and the calibration of future exploit decisions. A strong crawler therefore needs distribution-aware bookkeeping, not just pointwise priorities. ⁵

The translation is strongest when the crawler treats each URL as a latent-state bundle rather than a page to be greedily fetched. The "range" for a frontier URL should include posterior probabilities over content class, value density, structural role, duplication, freshness hazard, crawlability, compliance cost, and the value of descendants out to depth n . Chakrabarti and colleagues framed focused crawling as a "leap of faith" from a source page to an unseen target, using source-page clues to predict target worth; more recent neural-prioritization work similarly propagates quality estimates through local link structure. Hawrilenko's range-thinking and these crawl papers point to the same design rule: infer **futures**, not just immediate relevance. ⁶

Poker concept	Hawrilenko or primary basis	Crawler mechanism	Frontier translation
Range construction	Hawrilenko emphasizes knowing your own hand/distribution and updating frequencies street by street. ⁷	Posterior over latent page/site states	For each URL u , maintain $p(z_h x_u, G_t)$ for $h = 1..n$, where z_h includes value, freshness, authority, novelty, trap risk, duplication, and cost.
Hand reading	Hawrilenko argues against brittle exact reads and in favor of distributional reasoning. ⁸	Predict unseen target/page-section properties from local evidence	Infer target value from parent text, DOM/link context, anchor text, URL path, host history, robots/sitemap metadata, and neighborhood embeddings.
Pot odds	He replaces "am I good often enough?" with "how often must I continue to make bluffing indifferent?" and gives the $1/(1 + s)$ continuation logic. ⁹	Marginal continuation threshold	Continue crawling a branch when expected marginal utility per request exceeds the opportunity-cost threshold set by backlog, latency, and budget pressure.

Poker concept	Hawrilenko or primary basis	Crawler mechanism	Frontier translation
Bet sizing	Hawrilenko ties bet frequency to size; larger bets require fewer calls and different bluff ratios. ¹⁰	Resource allocation size	“Size” becomes the amount of budget allocated to a branch: number of concurrent fetches, revisit rate, parser depth, or follow-link beam width. Bigger allocations require stronger expected edge.
Bluffing ratio	He links value-bluff balance to sizing and warns against getting “out of whack.” ¹¹	Exploration budget ratio	Maintain a bounded exploration fraction for uncertain candidates; increase it only when model uncertainty is informative and trap hazard is controlled.
Aggression	Hawrilenko’s exploit-downstream idea is to enlarge the pot earlier when later folds are likely. ¹²	Proactive branch expansion	If a site historically yields high-value descendants two hops later, allocate more early requests upstream so later discoveries are more valuable.
Exploitative vs GTO play	Hawrilenko says to start from balance/unexploitable play and “expand to the margins” as reads strengthen. ¹³	Base policy plus bounded site-specific overrides	Score with a calibrated default policy, then apply a clipped exploit bonus/penalty using host- or template-level evidence and confidence.
Fold equity	He describes exploiting players who fold too much by expanding marginal bluffs. ¹³	Probe equity	Use low-cost, compliant probes —robots, sitemap, feed discovery, HEAD, lightweight HTML classification—when they are likely to reveal enough to unlock or prune a subtree.
Blockers	Hawrilenko’s card-removal logic tracks how observed cards remove combinations. ¹⁴	Structural blockers	Canonicals, noindex, duplicate-signature hits, forbidden paths, query-parameter patterns, and known site-template rules eliminate or reduce possible high-value futures.

Poker concept	Hawrilenko or primary basis	Crawler mechanism	Frontier translation
ICM	MIT's ICM lecture notes emphasize payout-aware chip value, bubble sensitivity, and stage-dependent aggression. ¹⁵	Mission-stage utility reshaping	Near deadlines or must-hit objectives, shift from raw expected pages to "payout equity": probability of covering critical topics, fresh pages, or high-authority sources before cutoff.
Risk management	Hawrilenko warns that reads are usually overestimated and that extreme deviations get punished. ¹⁶	CVaR-aware frontier control	Penalize high-variance branches, particularly those with severe trap, latency, or blocking downside; only widen the exploit band with sufficient evidence.
Opponent modeling	He notes the other side is also modeling you; this creates a leveling game. ¹⁷	Adversarial site behavior model	Estimate site response functions: rate-limit propensity, honeypot risk, dynamic trap likelihood, duplicate expansion, redirect loops, and parser-hostility.

Some concepts require extension beyond Hawrilenko's own published material. **ICM** is tournament theory from the broader MIT poker curriculum rather than his specific lecture, and **blockers** are best read as a crawler-side analogy derived from his card-removal and combination-count reasoning. That is still faithful to the spirit of the translation: observed evidence changes the support of the latent state space, and optimal decisions should respond to those support shifts. ¹⁸

A crucial design implication follows from Hawrilenko's warning against overconfidence. The crawler should almost never make all-or-nothing site judgments from a single page, just as he argues a strong read should move the margins only a little unless confidence is truly high. In practice, this means exploit adjustments should be **confidence weighted**, sample-size aware, and clipped. A site that looks promising after one lucky page should receive a mild boost, not a branch takeover. ¹³

Frontier management algorithm

The crawler should separate **policy correctness**, **scheduler correctness**, and **hardware correctness**. Policy correctness comes from the value model and bounded exploit layer. Scheduler correctness comes from the URL-frontier, host politeness, robots compliance, and duplicate control described in Mercator and related crawling literature. Hardware correctness comes from keeping any quantum component small enough to fit real shot, instruction-memory, and latency budgets on current platforms. ¹⁹

A practical score for candidate URL u at time t is:

$$\text{Score}(u, t) = \underbrace{\sum_{h=1}^n \gamma^{h-1} \mathbb{E}[V_h(u) \mid x_u, G_t]}_{\text{multi-hop value}} - \lambda_c \mathbb{E}[C(u)] - \lambda_l \mathbb{E}[L(u)] - \lambda_r \text{CVaR}_\alpha(R(u)) + \beta_t U(u) + \eta_t E(u)$$

where V_h is expected valuable information at hop h , C is crawl cost, L is delay/latency cost, R is downside risk, U is an uncertainty bonus for exploration, and E is a bounded exploit term learned from host-level or template-level behavior. The parameters $\gamma, \lambda_c, \lambda_l, \lambda_r, \beta_t, \eta_t$ are tunable and can be stage dependent. The Hawrilenko translation sits mainly in the **bounded** exploit term and in the rule that exploration/exploitation should be managed by percentiles and margins, not by absolute anecdotes. ²⁰

A useful decomposition is:

$$E(u) = \text{clip}(w^\top \Delta_{\text{site}}(u), -\delta(\sigma_u, m_u), +\delta(\sigma_u, m_u))$$

where $\Delta_{\text{site}}(u)$ measures deviations between the site's observed yield and the global baseline, σ_u is predictive uncertainty, and m_u is effective sample size. The exploit band δ should widen only as uncertainty falls and local evidence accumulates. This is the crawler analogue of Hawrilenko's "small read, move the margins a little; big read, move them more." ¹³

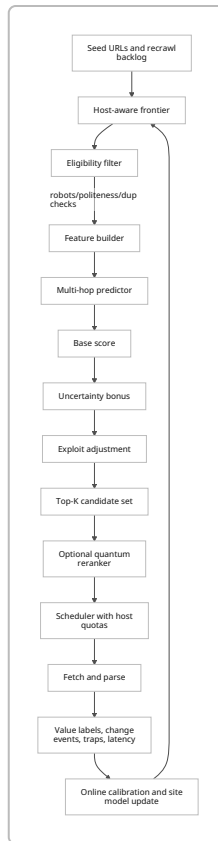
The "bet-sizing" analogue is a discrete resource-allocation action $a_u \in \{0, 1, 2, 4, \dots\}$ representing requests or concurrency units granted to a branch over the next scheduling window. A simple stage-aware choice is:

$$a_u = \arg \max_{a \in \mathcal{A}} \frac{\mathbb{E}[\Delta V(u; a)]}{a^\rho} \quad \text{s.t. politeness, host caps, and budget constraints}$$

with $\rho > 1$ to reflect diminishing returns. Large "bets" should be rare and reserved for candidates in the top value percentiles or for branches with strong two-hop/three-hop predicted spillover. This is the crawler analogue of not inflating the pot without sufficient range advantage. ²¹

"Bluffing" in this context should never mean deceptive or noncompliant behavior. It should mean **low-cost, standards-compliant probes** that reveal whether a path is worth a larger commitment. Examples include robots retrieval, sitemap discovery, feed discovery, HEAD requests where appropriate, DOM-lite parsing, or fetching a category page before expensive descendants. Since the Robots Exclusion Protocol is standardized and explicit about crawler controls, these probes must remain within the allowed surface. ²²

Adversarial site behavior belongs directly in the opponent model. Stanford's IR material notes that crawlers must be robust to spider traps and other malicious server behavior, while crawling lectures also treat politeness and trap avoidance as core requirements. The frontier policy should therefore estimate $p_{\text{trap}}(u)$, $p_{\text{block}}(u)$, $p_{\text{duplicate-farm}}(u)$, and $p_{\text{rate-limit}}(u)$, and discount scores when those hazards are high. A branch that expands URL count faster than information yield should be treated like an opponent overbluffing a line that gives away its imbalance. ²³



Pseudocode

Inputs:

Seeds S_0
 Horizon n
 Global request budget B
 Host politeness caps H
 Frontier F

Models:

M_{value} -> predicts multi-hop value by hop
 M_{cost} -> predicts fetch and parse cost
 M_{fresh} -> predicts change hazard / freshness urgency
 M_{risk} -> predicts trap, duplicate, block, latency hazards
 M_{site} -> estimates host/template deviations from baseline
 Q_{rerank} -> optional quantum or quantum-inspired reranker for top-K only

Initialize:

Enqueue seeds with neutral prior
 For each host h : $\text{quota}[h]$ = politeness-compliant request schedule
 site_stats = empty
 $\text{budget_remaining} = B$

```

While budget_remaining > 0 and F not empty:
    eligible = {u in F : robots_allowed(u) and host_quota_available(host(u))}
    if eligible is empty:
        advance scheduler clock
        continue

    For each u in eligible:
        x_u = build_features(u, parent_context, host_history, graph_state,
time_state)
        posterior_u = infer_latent_range(x_u, n)
        EV_u = sum_{h=1..n} gamma^(h-1) * E[V_h | posterior_u]
        Cost_u = E[C | x_u]
        Risk_u = CVaR_alpha(R | x_u)
        Explore_u = beta_t * uncertainty_bonus(posterior_u, visit_counts(host(u)))
        Exploit_u = clipped_site_adjustment(M_site, host(u), template(u),
confidence_u)
        Fresh_u = freshness_urgency(M_fresh, u, host(u))
        Score_u = EV_u + Fresh_u + Explore_u + Exploit_u - lc*Cost_u - lr*Risk_u

    topK = host_diversified_select(F, Score, K)

    if quantum_enabled:
        q_features = compress(topK.features, d <= 12)
        q_scores = Q_rerank(q_features, shots, depth, coupling_constraints)
        selected = merge_classical_quantum(topK, q_scores, host_quotas)
    else:
        selected = schedule_by_score(topK, host_quotas)

    For each selected u:
        response = fetch(u)
        budget_remaining -= request_cost(response)
        update_host_quota(host(u))
        if blocked_or_forbidden(response):
            update_site_model_negative(host(u), u)
            continue

        page = parse(response)
        labels = derive_training_signals(page, downstream_yield, freshness_event,
trap_event)
        update_models_online(labels)
        outlinks = extract_links(page)
        For each v in outlinks:
            if unseen_or_better_update(v):
                enqueue_or_update(F, v, parent=u)

    recalibrate_if_needed()

```

The nonnegotiable constraint is that the frontier manager must remain **host aware**, not just score aware. Mercator’s design explicitly separated the ordering problem from the host-level politeness problem by assigning URLs through host-sensitive subqueues, and RFC 9309 formalizes robots controls. Any Hawrilenko-style aggression that ignores these constraints stops being strategy and becomes a systems bug. ²⁴

Predictive models for multi-hop estimation

The predictive problem is not a single scalar ranking task. It is a **multi-task, partially observed, dynamic-graph forecasting problem**: estimate what a page and its descendants are likely to look like up to n hops before those nodes are fetched. The closest crawler literature already assumes that unseen-target value can be inferred from source-page clues, anchor text, and local topology; Hawrilenko adds the discipline to treat these predictions as **ranges with support and uncertainty**, not certainty claims. ²⁵

A strong feature schema should combine lexical, structural, historical, graph, and operational signals. At minimum, features should include URL path and query-token patterns; anchor text and nearby DOM context; parent-page semantic embedding; host/domain/site-section identity; inlink count seen so far; canonical/noindex/robots/sitemap/feed signals; content type; fetch latency and byte size priors; freshness/change-rate history; redirect behavior; duplicate fingerprint history; and host-level yield, trap, and block rates. Cho and Garcia-Molina’s refresh framework shows why change-rate and page importance belong in the same scheduling model, and modern temporal graph models justify treating these signals as time-evolving rather than static. ²⁶

Three model families are appropriate.

Classical baseline

A tree ensemble remains the best low-risk first deployment. Gradient-boosted trees handle heterogeneous tabular features, missingness, and nonlinear interactions well, and they are easy to calibrate. Pair a host-aware GBDT ranker with per-task heads for expected value, freshness hazard, latency, duplicate risk, and trap risk. Add deep-ensemble replication for uncertainty, because deep ensembles are simple, scalable, and empirically strong for calibration and domain-shift awareness. ²⁷

Dynamic graph model

For higher performance, use an inductive graph encoder such as GraphSAGE or GAT for local neighborhood aggregation, then a Temporal Graph Network for event-time updates. GraphSAGE is attractive because crawlers constantly see unseen nodes and evolving subgraphs; GAT is attractive because not all neighbors deserve equal weight; TGN is attractive because the crawl graph and page-state graph evolve continuously, and TGN explicitly models memory and timestamped events. This family is the most natural fit for n -hop attribute estimation because it can learn from both topology and temporal feedback. ²⁸

Quantum or hybrid quantum model

The honest near-term role of quantum ML is **augmentation**, not replacement. Primary quantum-search and ranking work suggests that quantum-walk-based ranking can surface secondary hubs and break degeneracies in ways that differ from classical PageRank, while quantum kernels and variational classifiers

offer alternative ways to classify compressed feature representations. Hybrid quantum graph work and the 2026 A2QTGN preprint suggest that small quantum modules can enrich dynamic graph representations, but this remains experimental and should be treated as a research branch, not the default production path.

29

The practical implication is that the quantum stage should rerank only the top few dozen or few hundred candidates after a classical filter. Current IBM platform documentation makes clear that execution budgets are finite and hardware constraints matter, including shot counts and instruction-memory limits. That pushes the quantum step toward compressed low-dimensional candidate pools, shallow circuits, and batch reranking rather than full-frontier evaluation.

30

Recommended model stack and starting hyperparameters

Layer	Recommendation	Starting hyperparameters	Why
Base value model	LightGBM/XGBoost ranker, multi-task heads	500–1000 trees, depth 6–8, learning rate 0.03–0.07, subsample 0.8, colsample 0.8	Strong on heterogeneous crawler features; fast retraining.
Graph encoder	GraphSAGE or GAT	3 layers, hidden 256, fanout [15,10,5], dropout 0.1, residual on	Inductive unseen-node support and locality-aware aggregation. 31
Temporal layer	TGN	Memory 256, time-encoding 64, message MLP 2×256, negative ratio 5:1	Designed for dynamic event graphs and edge prediction. 32
Uncertainty	Deep ensemble	5 members, independent seeds, temperature scaling on validation	Well-calibrated uncertainty under shift. 27
Multi-hop decoder	Shared trunk + hop-specific heads	$n \in \{1, 2, 3, 4\}$, default $n = 3$, discount $\gamma = 0.6$ –0.85	Explicitly estimates descendant value decay.
Exploit module	Hierarchical empirical-Bayes site model	Min 20 observations before nontrivial exploit, clip at 5–10% score shift initially	Implements “expand only at the margins.” 13
Quantum reranker	Quantum kernel SVM or shallow VQC on compressed top- K	top- K =32–64, PCA/autoencoder to 8–12 dims, 2–4 reupload layers, 2048–8192 shots	Small-set reranking is the most credible NISQ role. 33

Training labels should be multi-task. The key supervision targets are: immediate page value, cumulative value discovered within n hops, freshness/change event within a time window, authority proxy, answer-support or valuable-information label, novelty gain, duplicate flag, trap flag, and realized request cost. A label schema that distinguishes “page was relevant” from “page led to valuable descendants” is especially important, because the whole point of the Hawrilenko translation is to reason over **line value**, not just current-board value.

34

Experimental design and datasets

The evaluation should be built around the proposition that **frontier ordering matters most under incomplete crawls**. Stanford’s URL-ordering paper showed exactly that: when resources are limited and revisits are necessary, getting important pages first is materially better than arbitrary ordering. Focused-crawling work formalized harvest rate and loss rate; recent neural crawling work adds early-stage ranking metrics and search-effectiveness evaluation on ClueWeb22-derived subsets. Those three traditions should be combined in one experimental suite. ³⁵

Datasets

For web-scale structure and content, use **Common Crawl** and **ClueWeb22**. Common Crawl provides open web-crawl data, and both Common Crawl and Web Data Commons expose large hyperlink graphs suitable for authority and reachability experiments. ClueWeb22 is particularly attractive because it includes HTML, clean text, screenshots, and inlink/outlink data, and it was designed to support retrieval and web research. For realistic query/value judgments, pair ClueWeb22 with **MS MARCO Web Search**, which uses ClueWeb22 as its corpus and provides millions of clicked query-document labels. For multi-perspective “valuable information” tasks, add **Researchy Questions**, which releases question logs and clicked ClueWeb22 URLs. For dynamic-graph methodology and temporal link prediction, use **Temporal Graph Benchmark** datasets for controlled temporal ablations and transfer experiments. ³⁶

Metrics

The primary metrics should be:

- **Coverage** of valuable pages and site sections under fixed budget.
- **Harvest rate** and **loss rate** for valuable-information discovery.
- **Weighted freshness** and **age**, following the Cho–Garcia-Molina framework.
- **Time to first valuable page** and **area under the harvest curve**.
- **Precision/recall/F1** for valuable-information prediction and descendant yield.
- **nDCG / maxNDCG / downstream search effectiveness** on held-out queries where appropriate.
- **Cost per valuable page**, **cost per fresh page**, and total bytes, requests, and wall-clock latency.
- **Robustness** under injected spider traps, duplicates, rate limits, and adversarial templates. ³⁷

Baselines

The baseline suite should include FIFO **BFS**, **DFS**, classic **best-first** with parent relevance, **PageRank-informed** ordering where the local graph is large enough, **OPIC/adaptive OPIC**, focused crawling with source-page relevance propagation, a freshness-aware recrawl scheduler from the Cho–Garcia-Molina family, and a modern neural semantic-prioritization baseline such as the parent-quality propagation paradigm used in recent neural crawling work. If the quantum reranker is tested, compare it against a classical reranker of equal input dimensionality and latency budget. ³⁸

Simulation setup

The cleanest testbed is an **offline replay simulator** over a fixed crawl corpus plus temporal metadata. Build a directed graph from ClueWeb22 or Common Crawl. Reveal nodes only when “fetched.” Attach

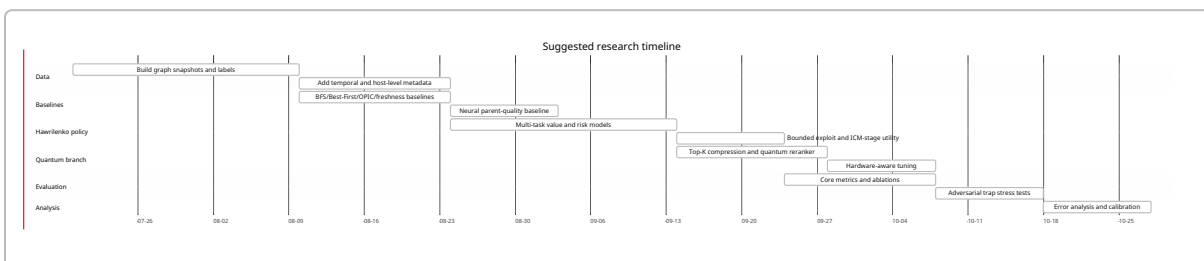
timestamped change events where available or estimated from recrawl traces. Use host-level request-delay and robots controls. Then inject adversarial motifs: calendar expansions, faceted search loops, query-parameter explosions, duplicate mirrors, redirect mazes, honeypot leaves, and hosts whose apparent value decays sharply after the first page. Stanford IR materials explicitly identify spider traps as a robustness requirement, so these should be first-class test conditions rather than afterthoughts. 39

Ablation studies

The ablations should test the Hawrilenko translation directly:

1. Remove the **distribution-aware rank percentile** and use only point estimates.
2. Remove the **bounded exploit term** and keep only the base policy.
3. Allow unbounded exploit updates to measure overconfidence damage.
4. Remove **uncertainty bonuses** and use greedy exploitation only.
5. Remove **card-removal/blocker features** such as canonical, duplicate, and query-pattern eliminators.
6. Remove **ICM-like stage-aware utility** and use a stationary objective.
7. Remove the **quantum reranker** and compare the same model stack classically.
8. Vary n to see whether the gain really comes from multi-hop reasoning or just one-hop relevance.

40



Visualization should focus on **curves**, not just endpoint numbers. The most revealing plots will be harvest-rate-over-budget curves, freshness-age tradeoff fronts, time-to-first-value survival plots, calibration curves for uncertainty, site-level exploit adjustments over time, and Sankey-style flow diagrams showing how many URLs are filtered by blockers, robots, duplicate control, trap model, and politeness scheduler before fetching. Recent neural crawling work uses early-stage metrics because that is exactly where prioritization matters most. 41

Failure modes and practical recommendations

The first failure mode is **range collapse**: the model becomes too sure that a branch is valuable or worthless after too little evidence. This is the crawler form of Hawrilenko's warning that players usually overestimate their read strength. In practice, range collapse shows up as "one good page from a host causes domain flooding" or "one bad template causes permanent host neglect." The cure is ensemble uncertainty, exploit clipping, and minimum evidence thresholds before widening margins. 42

The second failure mode is **mistaking local graph evidence for global authority**. Cho, Garcia-Molina, and Page already showed that ordering metrics matter, but later work and recent neural crawling authors also note that PageRank-style importance from partial graphs can be misleading. This makes a purely local

graph-importance policy dangerous, especially early in a crawl. The fix is to combine local graph features with semantic and host-history features and to treat local authority as one uncertain feature among many, not the final truth. ⁴³

The third failure mode is **tunnel vision from over-exploitation**. Hawrilenko’s “exploit downstream” idea is useful only when paired with balance. If a crawler keeps “making the pot bigger” on branches that once paid off, it may starve exploration and miss emerging or fresh content elsewhere. This is why the policy should explicitly reserve exploration budget and why ICM-like stage utility should be dynamic: early in the crawl, exploration is more like chip accumulation; late in the crawl, certainty around must-hit targets matters more. ⁴⁴

The fourth failure mode is **adversarial and pathological site response**. Spider traps, duplicates, and rate limits can dominate the request budget if not modeled. The crawler literature is explicit that robustness and politeness are mandatory, not optional. Therefore, trap-hazard estimation, duplicate fingerprints, canonical consolidation, robots caching, and host-aware queueing should sit below the model and be enforced even when the score says “go.” ⁴⁵

The fifth failure mode is **quantum overhead overwhelming any ranking benefit**. The quantum literature cited here is promising but mostly about ranking, kernels, or graph-learning experiments, not production crawl control. A top-*K* reranker that adds tens or hundreds of milliseconds and substantial operations cost but improves no retrieval or harvest metric is not strategic aggression; it is spew. The safe recommendation is to make the quantum branch optional, latency-capped, and benchmarked strictly against equally expensive classical rerankers. ⁴⁶

The final recommendation is therefore conservative and rigorous. Build the crawler around a **classical, host-aware, uncertainty-calibrated frontier**. Let the Hawrilenko contribution define the policy philosophy: reason over distributions, keep the default policy balanced, exploit only at the margins, and resize commitment only when the future line justifies it. Then add a hybrid quantum layer only where it has the best chance to matter—small candidate-set reranking or compact subgraph scoring—while keeping the entire system compliant with robots, politeness, and bounded host concurrency. That architecture best respects both the poker theory and the web-crawling literature. ⁴⁷

¹ ² ⁵ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁶ ¹⁷ ¹⁸ ²⁰ ²¹ ⁴⁰ ⁴² ⁴⁴ ocw.mit.edu

https://ocw.mit.edu/courses/15-s50-poker-theory-and-analytics-january-iap-2015/269df0542eb50a94faeaf0286cfa959e_kn92WXcKr0M.pdf

³ ¹⁹ ²⁴ ⁴⁷ cs.ucr.edu

<https://www.cs.ucr.edu/~vagelis/classes/CS242/publications/scalable-crawler.pdf>

⁴ ³⁵ ³⁸ ⁴³ Efficient Crawling Through URL Ordering

<https://www.ra.ethz.ch/CDstore/www7/1919/com1919.htm>

⁶ ²⁵ ³⁴ ³⁷ cse.iitb.ac.in

<https://www.cse.iitb.ac.in/~soumen/doc/www2002m/p336-chakrabarti.pdf>

¹⁵ Lecture 6: Independent Chip Model

https://ocw.mit.edu/courses/15-s50-how-to-win-at-texas-holdem-poker-january-iap-2016/5a8392e939175758331a28be9a64645c_MIT15_S50IAP16_L6.pdf

22 RFC 9309: Robots Exclusion Protocol

https://www.rfc-editor.org/rfc/rfc9309.html?utm_source=chatgpt.com

23 39 45 20 Web crawling and indexes - Stanford University

https://nlp.stanford.edu/IR-book/pdf/20crawl.pdf?utm_source=chatgpt.com

26 cui.unige.ch

<https://cui.unige.ch/tcs/cours/algoweb/2006/articles/crawler.pdf>

27 Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles

https://arxiv.org/abs/1612.01474?utm_source=chatgpt.com

28 31 Inductive Representation Learning on Large Graphs

https://arxiv.org/abs/1706.02216?utm_source=chatgpt.com

29 46 [1511.04823] Comparing classical and quantum PageRanks

<https://arxiv.org/abs/1511.04823>

30 Job limits | IBM Quantum Documentation

https://quantum.cloud.ibm.com/docs/guides/job-limits?utm_source=chatgpt.com

32 Temporal Graph Networks for Deep Learning on Dynamic Graphs

https://arxiv.org/abs/2006.10637?utm_source=chatgpt.com

33 [1804.11326] Supervised learning with quantum enhanced feature spaces

<https://arxiv.org/abs/1804.11326>

36 Common Crawl - Open Repository of Web Crawl Data

https://commoncrawl.org/?utm_source=chatgpt.com

41 Neural Prioritisation for Web Crawling

<https://arxiv.org/html/2506.16146v2>